

动态非对称双向 Labeling 在软时间窗函数标签中的实现思路

2026 年 5 月 26 日

1 Tilk 动态 half-way point 的基本思路

传统 bounded bidirectional labeling 通常先选一个固定的 half-way point，例如把时间资源区间 $[L, U]$ 的中点作为分界点。Forward labels 只向右扩展到该点附近，backward labels 只向左扩展到该点附近，最后再将两侧 labels 合并。这个做法的问题是：forward 和 backward labeling 的工作量通常并不对称。固定取 $H = (L + U)/2$ 时，一侧可能很快结束，另一侧却产生大量 labels。

Tilk et al. 提出的 dynamic half-way point 不再固定一个唯一的中间点，而是维护两个动态边界 H_B 和 H_F ，并始终保持 $H_B \leq H_F$ 。其中， H_B 可以理解为 backward side 不需要再向左越过的边界， H_F 可以理解为 forward side 不需要再向右越过的边界。初始时设置 $H_B = L$ 、 $H_F = U$ ，表示一开始两边都没有被固定中点限制。随着 forward 和 backward labels 被处理， H_B 单调上升， H_F 单调下降，两边逐步向中间收缩。

具体地，设 forward label 的单调资源值为 r_F ，backward label 的单调资源值为 r_B 。若处理一个 forward label 且 $r_F \leq H_F$ ，则该 label 可以继续扩展，并在处理后更新 $H_B \leftarrow \max\{H_B, \min\{r_F, H_F\}\}$ 。在通常情形下，这等价于将 H_B 推进到已处理 forward label 的资源位置。反过来，若处理一个 backward label 且 $r_B > H_B$ ，则该 label 可以继续扩展，并在处理后更新 $H_F \leftarrow \min\{H_F, \max\{r_B, H_B\}\}$ 。在通常情形下，这等价于将 H_F 推进到已处理 backward label 的资源位置。

这里使用 min 和 max 的目的，是保证两个边界始终满足 $H_B \leq H_F$ 。如果一开始令 $H_B = H_F = H$ ，该机制就退化为传统的静态 half-way point。因此，静态 midpoint 可以看作 dynamic half-way point 的一个特殊情形。

因为 forward labels 按资源非降序处理，所以当你处理到 r_F 时，所有更小资源的 forward labels 已经被处理过了。于是左侧区域已经由 forward 侧覆盖，backward 再往左做，会产生很多重复或低价值的工作，但这个和按资源顺序处理关系也不大，主要还是他这个更新的方式。

2 Tilk 动态双向 labeling 的求解流程

Tilk-style dynamic half-way point 的关键不只是更新 H_F 和 H_B ，还包括 forward 和 backward 两侧的处理顺序。它不是先完整生成一侧 labels，再生成另一侧 labels；而是两侧同时维护未处理 label 队列，每一步选择一侧处理一个 label。

一个典型流程如下：

1. 选择一个单调资源区间 $[L, U]$ ，初始化 $H_B = L$ 、 $H_F = U$ 。
2. 初始化 source 侧 forward label 和 sink 侧 backward label，并分别放入 forward 和 backward 队列。

3. 每一步通过某个规则选择处理 forward 还是 backward。文献中常用的规则是选择当前未处理 labels 较少的一侧，以平衡两边的工作量。
4. 如果选择 forward，则弹出资源值最小的 forward label。若其资源值不超过 H_F ，则沿 outgoing arcs 扩展该 label，并更新 H_B ；否则该 label 不再继续扩展。
5. 如果选择 backward，则弹出资源值最大的 backward label。若其资源值大于 H_B ，则沿 incoming arcs 扩展该 label，并更新 H_F ；否则该 label 不再继续扩展。
6. 当两侧队列都为空后，对两侧保留下来的 labels 执行 merge 或 join，生成完整路径。

这个流程要求 label 的出队顺序与单调资源一致：forward labels 应按资源非降序处理，backward labels 应按资源非升序处理。否则，处理到一个资源值较大的 forward label 并不意味着所有更小资源值的 forward labels 都已处理完成，此时用它更新 H_B 就不可靠。

3 用户当前想法

下面保留当前讨论中的原始想法，作为后续代码实现的目标说明。

OK，这个东西应该在我这个软时间窗 VRP 上边也可以使用吧，即可以这么做，之前不能做是因为正反 label 的函数的定义域只使用了整个 $C_{\max}$ 的一半的定义域希望做一些加速，那其实这个只是为了加速，我们还是可以保持正反向的 label 是全定义域的，不做一半的裁剪。这时候应该就可以做他这种非对称的双向 labeling 算法了，即利用的资源是正向的最早完成时间和反向的最迟完成时间，利用这两，如果正向的扩展后超过 H_F 的时间了，这个扩展就不要了，反向的同理，最迟完成时间小于 H_B 了就删掉。正向的基于最早完成时间单调递增出队，反向的基于最晚完成时间递减出队。

最坏情况下其实无非就是，初始设置 $H_B = H_F$ ，此时扩展函数的时候保留全部定义域就好了。相对之前那种可能在函数操作，占优上会差一些。但不知道会差多少。这个可能也需要测试一下。

4 在软时间窗函数标签中的改造方式

在当前软时间窗问题中，label 不是一个标量状态，而是一个关于完成时间的分段线性函数。现有实现为了加速，把 forward label 的函数定义域裁到 $[0, T^{\text{mid}}]$ ，把 backward label 的函数定义域裁到 $[T^{\text{mid}}, H]$ 。这种半域裁剪对函数运算和 dominance 都有帮助，但也使得 midpoint 成为函数定义域的一部分。一旦想动态移动 half-way point，就会遇到函数切分、缓存失效和 dominance graph 重建等问题。

如果要引入 Tilk-style dynamic half-way point，一个更自然的方案是：不再把函数定义域裁成两半，而是让正反向 labels 都保留完整有效定义域。此时 H_F 和 H_B 不再用于裁剪函数定义域，而只用于控制 label 是否继续扩展。

具体地，forward label 仍表示某个前缀路径在不同完成时间下的最优 reduced-cost frontier。它的单调资源可以取该函数定义域的左端点，即该前缀最早可能完成时间，记为 $r_F(L) = \ell_L$ 。随着路径向后扩展，最早完成时间不会下降，因此这是一个单调递增资源。Backward label 对称地表示某个后缀在不同接入时间下的 reduced-cost frontier，

其单调资源可以取该函数定义域的右端点，即该后缀允许的最晚完成或接入时间，记为 $r_B(L) = \rho_L$ 。随着 backward label 向前扩展，最晚可接入时间不会上升，因此这是一个单调递减资源。

基于这两个资源，可以将 Tilk 的动态边界直接用于软时间窗函数标签：forward 队列按 ℓ_L 从小到大出队，backward 队列按 ρ_L 从大到小出队。若 forward 扩展后得到的 child label 满足 $\ell_{L'} > H_F$ ，则该 child 已越过当前 forward 边界，不再生成或插入 forward label table；若 backward 扩展后得到的 child label 满足 $\rho_{L'} \leq H_B$ ，则该 child 已越过当前 backward 边界，也不再生成或插入 backward label table。这样，超出边界的路径段由另一侧 labeling 负责覆盖。

需要强调的是，这里的边界判断只影响是否继续保留该方向的 label，并不改变函数 label 的定义方式。函数本身仍在完整有效时间域上递推、normalize 和 dominance。这样可以避免动态移动 midpoint 时不断切分函数定义域。

5 建议的实现流程

可以将新的动态双向 labeling 实现为一个独立的 pricing mode，而不是直接在当前半域版本上小改。建议流程如下。

5.1 初始化

设当前 pricing horizon 为 H 。初始化 $H_B = 0$ 、 $H_F = H$ 。Forward source label 使用完整有效定义域初始化，backward sink label 也使用完整有效定义域初始化。Forward 队列按最早完成时间升序排序，backward 队列按最晚完成时间降序排序。

5.2 方向选择

每一轮选择一侧处理一个 label。第一版可以采用简单规则：选择当前未处理 labels 较少的一侧。如果 forward 队列为空，则处理 backward；如果 backward 队列为空，则处理 forward。这个规则的目标不是精确预测运行时间，而是避免一侧过早承担全部 workload。

5.3 Forward 处理

弹出最早完成时间最小的 forward label。若该 label 的最早完成时间已经大于 H_F ，则不再扩展。否则，对所有允许的后继任务执行扩展。扩展前可先用 setup time 和 processing time 计算 child 的最早完成时间下界；如果该下界已经大于 H_F ，则直接跳过该扩展，不再构造 child 的分段函数。对未越界的扩展，正常执行函数 shift、add、normalize 和 dominance。处理完该 forward label 后，用它的资源值更新 H_B 。

5.4 Backward 处理

弹出最晚完成时间最大的 backward label。若该 label 的最晚完成时间已经不大于 H_B ，则不再扩展。否则，对所有允许的前驱任务执行扩展。扩展前可先估计 child 的最晚完成或接入时间上界；如果该上界已经不大于 H_B ，则直接跳过该扩展。对未越界的

扩展，正常执行函数递推、normalize 和 dominance。处理完该 backward label 后，用它的资源值更新 H_F 。

5.5 Join

当两侧队列都处理完后，再执行 join。由于正反向函数都保留完整定义域，join 阶段不再需要依赖固定 T^{mid} 的常数延拓。对于 crossing arc (i, r) ，直接将 forward frontier 经由 setup 和 processing time 平移后，与 backward frontier 在公共定义域上相加，并加上 crossing arc 的 reduced-cost 固定项。若合并后的最小 reduced cost 为负，则恢复完整序列并生成列。

如果仍希望更早发现负列，也可以在处理 backward label 时做 streaming join，但从实现和调试角度看，第一版建议先做完整 forward/backward 动态扩展后统一 join。这样更接近 Tilk 的流程，也更容易验证 correctness。

6 与当前半域版本的差异

当前半域版本的主要优势是函数定义域短，单次 shift/add/normalize 和 dominance 比较都更便宜；同时 forward/backward 的候选范围被固定 midpoint 直接裁掉。缺点是效率高度依赖 T^{mid} 的选择。当 T^{mid} 选得偏左时，forward labels 很少而 backward labels 很多；选得偏右时可能出现相反问题。

动态 full-domain 版本的优势是避免手动选择固定 midpoint，并能根据真实 forward/backward labeling 状态自适应调整边界。它还消除了半域函数裁剪带来的 single-point、constant extension 和 cache 边界问题。代价是函数定义域更长，单个 label 的函数运算更重；dominance 也可能变弱，因为需要在更大的定义域上比较函数。因此，这个方案是否更快不能只从理论判断，需要实验比较。

建议至少比较三个版本：第一，当前 half-domain static midpoint 版本；第二，full-domain static midpoint 版本，即初始化 $H_B = H_F = \tau$ ；第三，full-domain dynamic H_B, H_F 版本。第二个版本可以帮助分离 full-domain 函数带来的额外成本，第三个版本则用于观察动态边界是否足以抵消这些成本。

7 实现注意事项

第一，队列排序必须改为资源排序。Forward 不能继续按最小 reduced cost 出队，而应按最早完成时间出队；backward 应按最晚完成时间出队。否则 H_B 和 H_F 的更新不再表示两侧已覆盖到的位置。

第二，扩展越界的 child 可以直接不生成。也就是说，若 forward child 的最早完成时间已经超过 H_F ，或者 backward child 的最晚完成时间已经不大于 H_B ，则该 child 不需要进入该方向的 label table。这个处理建立在真正的动态双向流程上：越过边界的部分应由另一侧 labels 覆盖。

第三，动态边界不应进入 dominance key。Dominance 仍应基于 terminal job、可达集合、函数 frontier 以及后续可能加入的 ng-memory 等状态进行。 H_F 和 H_B 只是扩展控制参数，不是 label 状态的一部分。

第四，若后续加入 ng-set 或 DSSR，需要进一步区分真实 visited set、ng-memory set 和永久 reachable set。动态 half-way point 与 ng-set 可以并存，但二者控制的是不同维度：前者控制时间资源上的扩展边界，后者控制重复访问的放松范围。

8 结论

Tilk-style dynamic half-way point 可以迁移到软时间窗函数标签问题，但更合适的实现方式不是在现有半域函数定义域上动态移动 T^{mid} ，而是改成 full-domain function labels，并仅用 H_B, H_F 控制 label 是否继续扩展。该方案利用 forward 最早完成时间和 backward 最晚完成时间作为单调资源，forward 按最早完成时间升序处理，backward 按最晚完成时间降序处理。若扩展后的 child 已跨过对应动态边界，则不再生成该 label。

这个方案可能缓解固定 midpoint 导致的 forward/backward workload 不平衡，但会增加单个 label 的函数运算和 dominance 成本。因此，建议作为一个独立实现版本进行测试，并与当前 half-domain static midpoint 版本进行系统比较。

参考文献

- [1] Tilk, C., Rothenbächer, A.-K., Gschwind, T., and Irnich, S. (2017). Asymmetry matters: Dynamic half-way points in bidirectional labeling for solving shortest path problems with resource constraints faster. *European Journal of Operational Research*, 261(2), 530–539.